

Artificial Intelligence Laboratory Assignment

Complete Worked Solution

Question 1 Solution : A* Search Algorithm Dry Run

A* Search expands the node with the lowest $f(n) = g(n) + h(n)$ at every step, where $g(n)$ is the exact cost accumulated from the start node and $h(n)$ is the heuristic estimate of the remaining cost to the goal. The algorithm terminates the moment the goal node itself is popped from the Open List for expansion (guaranteeing optimality since the heuristic is admissible/consistent here).

Step-by-Step Iteration Table

Iter	Current Node	$g(n)$	$h(n)$	$f(n)=g+h$	Open List (after expansion)	Closed List
1	A	0	7	7	B($f=8$), C($f=8$)	{ }
2	B	2	6	8	C($f=8$), D($f=12$), E($f=6$)	{A}
3	E	4	2	6	C($f=8$), D($f=9$), G($f=6$)	{A, B}
4	G	6	0	6	C($f=8$), D($f=9$) — GOAL REACHED	{A, B, E}

Detailed Explanation of Each Iteration

Iteration 1 — Expand A: Start node A has $g=0$, $h=7$, $f=7$. It is expanded, generating neighbours B ($g=0+2=2$, $h=6$, $f=8$) and C ($g=0+4=4$, $h=4$, $f=8$). Both are pushed to the Open List; A moves to the Closed List.

Iteration 2 — Expand B: B and C are tied at $f=8$; B is chosen (processed first / alphabetical tie-break). Expanding B generates: C via $B \rightarrow g=2+3=5$ (worse than existing $g=4$ for C, so C is NOT updated); D $\rightarrow g=2+7=9$, $h=3$, $f=12$; E $\rightarrow g=2+2=4$, $h=2$, $f=6$. Open List becomes {C($f=8$), D($f=12$), E($f=6$)}. B moves to Closed List.

Iteration 3 — Expand E: E has the lowest $f=6$ in the Open List, so it is expanded next (even though C was discovered earlier, its $f=8$ is higher). Expanding E generates: C via $E \rightarrow g=4+3=7$ (worse than existing $g=4$, not updated); D via $E \rightarrow g=4+2=6$, which IS better than the previous $g=9$ for D, so D is updated to $f = 6+3 = 9$; G $\rightarrow g=4+2=6$, $h=0$, $f=6$. Open List becomes {C($f=8$), D($f=9$), G($f=6$)}. E moves to Closed List.

Iteration 4 — Expand G: G now has the lowest $f=6$ in the Open List and is selected for expansion. Since G is the goal node, the search terminates successfully.

Backtracking the parent pointers gives the optimal path: **A $\rightarrow$ B $\rightarrow$ E $\rightarrow$ G.**

Path Cost Verification

Edge	A $\rightarrow$ B	B $\rightarrow$ E	E $\rightarrow$ G	Total
Weight	2	2	2	6

Alternative candidate paths are all more expensive — e.g. $A \rightarrow C \rightarrow E \rightarrow G$ costs $4+3+2 = 9$, and $A \rightarrow B \rightarrow C \rightarrow E \rightarrow G$ costs $2+3+3+2 = 10$ — confirming that $A \rightarrow B \rightarrow E \rightarrow G$ is indeed optimal.

Final Result — Question 1

1. Optimal Path : $A \rightarrow B \rightarrow E \rightarrow G$

2. Total Path Cost : 6

Question 2 Solution : Minimax with Alpha-Beta Pruning Dry Run

Alpha-Beta pruning maintains two bounds while traversing the game tree: α (the best value MAX can guarantee so far) and β (the best value MIN can guarantee so far). A branch is pruned as soon as $\alpha \geq \beta$, because the parent will never allow that branch's value to affect the final decision. The tree is evaluated strictly left to right as instructed.

Step 1 — Left MIN Node (children: 3, 5, 6)

Entering the left MIN node with inherited bounds $\alpha = -\infty$, $\beta = +\infty$ from the MAX root.

Child Visited	Comparison	Updated β	α	Prune?
Leaf = 3	$\beta = \min(+\infty, 3)$	3	$-\infty$	No
Leaf = 5	$\beta = \min(3, 5)$	3	$-\infty$	No
Leaf = 6	$\beta = \min(3, 6)$	3	$-\infty$	No

No pruning occurs at this node since all three children must be compared (α never rises above $-\infty$ here to trigger a cut). **Left MIN node returns value = 3** to the MAX root.

Step 2 — Update at MAX Root

The MAX root receives 3 from the left subtree: $\alpha = \max(-\infty, 3) = 3$. This updated $\alpha = 3$ is now passed down into the right MIN node.

Step 3 — Right MIN Node (children: 9, 1, 2)

Entering the right MIN node with inherited bounds $\alpha = 3$, $\beta = +\infty$.

Child Visited	Comparison	Updated β	α	Prune?
Leaf = 9	$\beta = \min(+\infty, 9)$	9	3	No ($3 < 9$)
Leaf = 1	$\beta = \min(9, 1)$	1	3	YES — $\alpha(3) \geq \beta(1)$
Leaf = 2	Not visited — branch cut	—	—	PRUNED

After visiting leaf value 1, β drops to 1. Since α (3) is now $\geq \beta$ (1), the remaining child (leaf = 2) is **pruned** — it is never evaluated because MAX would never choose this branch over the already-guaranteed value of 3 from the left subtree. **Right MIN node returns value = 1** to the MAX root.

Step 4 — Final Decision at MAX Root

MAX root now compares its two children's backed-up values: $\max(3, 1) = 3$. Final α at root = 3.

Final Result — Question 2

- 1. Best Move for MAX :** Choose the LEFT branch (the MIN node whose backed-up value is 3)
- 2. Final Minimax Value :** 3
- 3. Pruned Nodes :** Right MIN node's third leaf (value = 2) — cut off by Alpha-Beta pruning

This solution was independently verified by running the accompanying Python program (search_algorithms.py), whose console trace exactly matches every value derived above. See output.png for the corresponding visual diagrams.